



PROJECT REPORT

Ghibli Garden

An Interactive, Studio-Ghibli-Themed Password Generator

Built with Python and Tkinter

Submitted by: Priyam Singh

B.Tech, Computer Science & Engineering (2024–2028)

Dr. K. N. Modi University

Document type: Source Code Walkthrough & Technical Report

Table of Contents

Table of Contents	2
1. Introduction	3
2. Objectives	3
3. Tools and Technologies Used	3
4. Program Structure Overview	3
5. Line-by-Line Code Walkthrough	5
5.1 Module Docstring and Imports (lines 1–14)	5
5.2 Colour Palette and Fonts (lines 16–40)	5
5.3 Class GhibliPasswordGenerator — Constructor (lines 43–64)	6
5.4 Painted Background: Sky, Hills, Sun and Clouds (lines 69–173)	7
5.5 Building the Storybook Panel (lines 178–280)	10
5.6 Password Generation and Strength Logic (lines 289–377)	13
5.7 Helper Class: RoundedCard (lines 380–409)	16
5.8 Helper Class: LeafButton (lines 412–443)	16
5.9 Program Entry Point (lines 446–448)	17
6. Feature Summary	18
7. Testing	18
8. Conclusion and Future Scope	19
References	19

1. Introduction

Passwords remain the first line of defence for almost every digital account, yet people often choose weak, memorable passwords because generating and remembering a strong, random one feels tedious. **Ghibli Garden** is a desktop application, written entirely in Python using the built-in Tkinter toolkit, that turns the otherwise dry task of password generation into a small, illustrated ritual. The interface is painted in the soft, hand-drawn style associated with Studio Ghibli films — watercolour skies, rolling meadow hills, drifting clouds, and a storybook-style card that hosts the actual controls — while the underlying logic uses Python's cryptographically secure secrets module to produce genuinely unpredictable passwords.

This report documents the complete source file, **ghibli_password_generator.py**, explaining what each part of the program does, in the order it appears in the file. It is intended as both a project report and a maintenance reference: a reader with basic Python knowledge should be able to follow the report and understand exactly how every visual element and every security decision in the program was implemented.

2. Objectives

- Generate strong, random passwords using a cryptographically secure random number source.
- Give the user fine-grained control over password composition — length, character sets, and exclusion of visually ambiguous characters.
- Present the tool through a distinctive, hand-crafted graphical interface rather than a generic form.
- Provide immediate, understandable feedback on password strength.
- Keep the entire application self-contained in the Python standard library, requiring no installation steps.

3. Tools and Technologies Used

Language	Python 3
GUI toolkit	Tkinter (including the Canvas, Frame, Entry, Scale, and Checkbutton widgets)
Randomness / security	secrets module (CSPRNG) for all password characters
Character sets	string module (ascii_uppercase, ascii_lowercase, digits)
Math utilities	math module (log2 for entropy, sin for hill-curve drawing)
Fonts	Georgia (headings/labels), Courier New (password display)

4. Program Structure Overview

The file is organised into one module-level configuration section and three classes:

- **Module level** — imports and a shared palette/font configuration used throughout the UI.
- **GhibliPasswordGenerator(tk.Tk)** — the main application window. It builds the background scenery, the foreground control panel, and holds all password-generation logic.
- **RoundedCard(tk.Frame)** — a reusable helper widget that draws a soft rounded-rectangle card, used as the storybook-style panel background.

- **LeafButton(tk.Canvas)** — a reusable helper widget that draws a pill-shaped, hand-drawn-feeling button with hover and click colour changes.

Section 5 below walks through the file from the first line to the last, in the same order the code is written, explaining every statement.

5. Line-by-Line Code Walkthrough

Each subsection below reproduces a block of the source file together with a table that explains the corresponding lines. Line numbers refer to **ghibli_password_generator.py** as submitted.

5.1 Module Docstring and Imports (lines 1–14)

Line	Code	Explanation
1	<code>#!/usr/bin/env python3</code>	Shebang line — allows the file to be executed directly on Unix-like systems as <code>./ghibli_password_generator.py</code> , telling the OS to run it with <code>python3</code> .
2–7	<code>"""Ghibli Garden ... """</code>	A module-level docstring describing the program and how to run it. It is purely documentation and has no effect on execution.
9	<code>import math</code>	Imported for <code>log2()</code> (used to estimate password entropy) and <code>sin()</code> (used to draw the wavy hill silhouettes).
10	<code>import random</code>	Imported but not used for password generation (that uses the <code>secrets</code> module instead); kept available for any non-security-sensitive randomness.
11	<code>import secrets</code>	Imported for cryptographically secure random choices — this is the module that actually generates the password's characters.
12	<code>import string</code>	Provides ready-made character-class constants: <code>string.ascii_uppercase</code> , <code>string.ascii_lowercase</code> , and <code>string.digits</code> .
13	<code>import tkinter as tk</code>	The standard GUI toolkit, aliased to <code>tk</code> for brevity; all widgets (<code>Tk</code> , <code>Frame</code> , <code>Canvas</code> , <code>Entry</code> , <code>Scale</code> , <code>Checkbutton</code> , <code>Label</code>) come from here.
14	<code>from tkinter import font as tkfont</code>	Imported for potential font introspection/measurement; the app mainly uses plain font tuples, but this import keeps the font submodule available.

5.2 Colour Palette and Fonts (lines 16–40)

Every colour used anywhere in the interface is defined once, here, as a named constant. Centralising the palette means the whole visual theme can be tuned by editing this one block.

Line	Code	Explanation
19–21	<code>SKY_TOP / SKY_MID / SKY_BOTTOM</code>	Three hex colours that form the vertical watercolour sky gradient, from pale blue at the top to warm cream near the horizon.
22–23	<code>MEADOW / MEADOW_DARK</code>	Greens used for the front and back meadow hill silhouettes, and for the tree/shrub shapes.
24–25	<code>PANEL / PANEL_EDGE</code>	The cream background colour of the storybook card and its darker tan border.
26–27	<code>INK / INK_SOFT</code>	The two text colours: <code>INK</code> for headings/important text, <code>INK_SOFT</code> for secondary captions.
28	<code>SUN</code>	The warm yellow used for the sun disc in the sky.

Line	Code	Explanation
29	CLOUD	Pure white, used for the drifting cloud shapes.
30–31	ROSE / ROSE_DARK	Base and hover colours for the “Copy” button.
32–33	BLUE_BTN / BLUE_BTN_DARK	Base and hover colours for the “Cast a New Spell” (generate) button.
34	TOTORO_GREY	A reserved grey tone kept in the palette for any future greyscale accents; not currently drawn on screen.
36–40	FONT_TITLE / FONT_LABEL / FONT_LABEL_B / FONT_MONO / FONT_SMALL	Font tuples of the form (family, size, weight). Georgia gives the storybook feel for text; Courier New (bold) is used only for the generated password so every character is easy to distinguish.

5.3 Class GhibliPasswordGenerator — Constructor (lines 43–64)

GhibliPasswordGenerator subclasses tk.Tk, so an instance of it is the application's main window.

Line	Code	Explanation
43	<code>class GhibliPasswordGenerator(tk.Tk):</code>	Declares the main window class, inheriting all standard Tk window behaviour (resizing, event loop, widget hosting).
44–45	<code>def __init__(self): / super().__init__()</code>	Constructor; first calls Tk's own initialiser so the underlying window/toolkit is ready before any customisation.
46	<code>self.title("Ghibli Garden – Password Generator")</code>	Sets the text shown in the OS window title bar.
47–48	<code>self.geometry(...) / self.minsize(...)</code>	Sets the initial window size to 600×760 pixels and forbids shrinking below 560×720, so the layout never becomes too cramped.
49	<code>self.configure(bg=SKY_TOP)</code>	Sets a fallback background colour (visible briefly before the painted canvas is drawn).
52	<code>self.length_var = tk.IntVar(value=16)</code>	A Tkinter integer variable bound to the length slider; default password length is 16 characters.
53–56	<code>self.upper_var / lower_var / digits_var / symbols_var</code>	Boolean variables bound to the four main character-set checkboxes, all defaulting to True (checked).
57	<code>self.avoid_ambiguous_var = tk.BooleanVar(value=False)</code>	Boolean variable for the optional “avoid look-alikes” checkbox, off by default.
58	<code>self.password_var = tk.StringVar(value="")</code>	Holds the currently displayed password; bound to the read-only Entry widget so updating it refreshes the display automatically.
59	<code>self.copied_var = tk.StringVar(value="")</code>	Holds the small confirmation caption shown after the Copy button is pressed.
61	<code>self._build_background()</code>	Builds the decorative scenery canvas (sky, hills, sun, clouds).
62	<code>self._build_panel()</code>	Builds the foreground storybook card containing every interactive control.
63	<code>self._float_clouds()</code>	Starts the recurring animation loop that drifts the clouds sideways.
64	<code>self.generate_password()</code>	Generates an initial password immediately, so the app never opens with an empty field.

5.4 Painted Background: Sky, Hills, Sun and Clouds (lines 69–173)

All decorative scenery is drawn on a single full-window `tk.Canvas` using simple shape primitives (rectangles, ovals and polygons) rather than image files, which keeps the program dependency-free and lets the scenery redraw itself cleanly at any window size.

5.4.1 Canvas setup and redraw trigger (lines 69–73)

Line	Code	Explanation
69–70	<code>def _build_background(self): / self.bg = tk.Canvas(...)</code>	Creates one <code>Canvas</code> widget, <code>self.bg</code> , that will hold every scenery element.
71	<code>self.bg.place(x=0, y=0, relwidth=1, relheight=1)</code>	Uses the <code>place</code> geometry manager to stretch the canvas to fill 100% of the window's width and height, and keeps it filling the window as it resizes.
72	<code>self.bg.bind("<Configure>", self._redraw_background)</code>	Whenever the canvas changes size (e.g. the user resizes the window), Tkinter fires a event, which is wired to redraw the entire scene at the new dimensions.
73	<code>self._cloud_ids = []</code>	An empty list that will later store the canvas item IDs of each cloud shape, so the animation loop can move them.

5.4.2 Redrawing the scene (lines 75–117)

Line	Code	Explanation
75–79	<code>def _redraw_background(self, event=None): ... w = ...; h = ...</code>	Clears every existing shape (<code>c.delete("all")</code>) and reads the canvas' current width/height, falling back to 560×720 if the canvas has not yet been laid out.
82–93	<code>steps = 60 / for i in range(steps): ...</code>	Draws the sky as 60 thin horizontal rectangles stacked from top to bottom, each coloured by blending between <code>SKY_TOP</code> → <code>SKY_MID</code> for the top 60% of the height, then <code>SKY_MID</code> → <code>SKY_BOTTOM</code> for the remaining 40% — producing a smooth watercolour gradient instead of a flat colour.
96–97	<code>c.create_oval(...) *2</code>	Draws the sun: a solid yellow disc near the top-right corner, plus a slightly larger, unfilled outline circle around it for a soft halo.
100–101	<code>hill_back = self._hill_points(...) / c.create_polygon(hill_back, ...)</code>	Computes a wavy hill outline positioned high up the canvas and fills it with a colour blended 35% toward the sky colour, so it reads as a hazy, distant hill.
103–104	<code>hill_front = self._hill_points(...) / c.create_polygon(hill_front, ...)</code>	Computes a second, lower and more pronounced wavy hill and fills it with the solid, saturated <code>MEADOW</code> colour so it reads as the near hill in front of the first one.
107–108	<code>self._draw_tree(...) / self._draw_shrub(...)</code>	Places one small tree near the left of the front hill and one round shrub cluster near the right, for a touch of storybook detail.
110–111	<code>self._cloud_ids = [] / for cx, cy, s in [...]: ...</code>	Draws three clouds at different relative positions and scales (a large one upper-left, a medium one upper-middle, a small one upper-right) and stores each cloud's shape IDs for animation.

Line	Code	Explanation
116–117	<pre>if hasattr(self, "panel_frame"): self.panel_frame.tkraise()</pre>	Because the background is redrawn on every resize (which would otherwise cover the foreground panel), this raises the storybook card back above the canvas after each redraw.

5.4.3 Colour blending helper (lines 119–126)

Line	Code	Explanation
119	<pre>@staticmethod</pre>	Marks <code>_blend</code> as not needing access to a class instance — it is a pure utility function grouped inside the class for organisation.
120	<pre>def _blend(hex1, hex2, t):</pre>	Takes two "#rrggbb" colours and a blend factor <code>t</code> between 0 and 1.
121–122	<pre> r1, g1, b1 = int(hex1[1:3], 16), ... / r2, g2, b2 = ...</pre>	Splits each hex string into its red, green and blue components and converts each two-character hex pair to an integer 0–255.
123–125	<pre> r = int(r1 + (r2 - r1) * t) / g = ... / b = ...</pre>	Performs simple linear interpolation on each channel independently: at <code>t=0</code> the result equals colour 1, at <code>t=1</code> it equals colour 2, and values in between are a smooth mix.
126	<pre> return f"#{r:02x}{g:02x}{b:02x}"</pre>	Formats the three resulting integers back into a zero-padded, lower-case hex colour string that Tkinter can use directly.

5.4.4 Wavy hill outline generator (lines 128–137)

Line	Code	Explanation
129	<pre>def _hill_points(w, h, base, amp, phase):</pre>	Generates the point list for one wavy hill polygon. <code>base</code> is the hill's average vertical position, <code>amp</code> is how tall the waves are, and <code>phase</code> shifts the wave horizontally so the front and back hills don't line up.
130	<pre> pts = [0, h]</pre>	The polygon starts at the bottom-left corner of the canvas, so the filled shape reaches all the way to the bottom edge.
131–135	<pre> n = 40 / for i in range(n + 1): ... y = base + amp * math.sin(...)</pre>	Samples 41 evenly spaced <code>x</code> positions across the canvas width; at each one, computes a <code>y</code> position that oscillates above and below <code>base</code> using a sine wave, creating the rolling-hill silhouette.
136	<pre> pts += [w, h]</pre>	Closes the shape by adding the bottom-right corner, so <code>create_polygon</code> can fill the whole area beneath the wavy line.
137	<pre> return pts</pre>	Returns the flat list of <code>x, y</code> coordinates ready to be passed to Tkinter's <code>create_polygon</code> .

5.4.5 Clouds, tree and shrub (lines 139–159)

Line	Code	Explanation
139–140	<pre>def _draw_cloud(self, c, x, y, scale=1.0): / r = 18 * scale</pre>	Defines a base radius for the cloud's puffs, scaled by the scale factor so clouds can be drawn at different sizes.

Line	Code	Explanation
142–147	<code>parts.append(c.create_oval(...)) ×3</code>	Draws three overlapping white ovals of different sizes and offsets — a wide flat oval as the cloud's body plus two rounder ovals for puffy tops — which together read as a single fluffy cloud.
148	<code>return parts</code>	Returns the list of the three canvas item IDs that make up this cloud, so the animation loop can move all of them together.
150–154	<code>def _draw_tree(self, c, x, y): ...</code>	Draws a small brown rectangle as the trunk, then three overlapping green ovals of decreasing offset as the leafy canopy.
156–159	<code>def _draw_shrub(self, c, x, y): ...</code>	Draws three overlapping green ovals of different sizes side by side to suggest a low, rounded bush.

5.4.6 Cloud drift animation (lines 161–173)

Line	Code	Explanation
161	<code>def _float_clouds(self):</code>	A self-scheduling method that produces the gentle, continuous horizontal drift of the clouds.
163–164	<code>try: / w = self.bg.winfo_width() or 560</code>	Reads the current canvas width (wrapped in try/except because the widget may not exist yet during shutdown).
165–167	<code>for group in self._cloud_ids: / for part in group: / self.bg.move(part, 0.3, 0)</code>	Nudges every shape that belongs to every cloud 0.3 pixels to the right on each tick.
168–170	<code>coords = self.bg.coords(part) / if coords and coords[0] > w + 60: / self.bg.move(part, -(w + 140), 0)</code>	Checks whether a cloud piece has drifted completely past the right edge; if so, jumps it back to just off the left edge, creating a seamless, endless drift.
171–172	<code>except tk.TclError: / pass</code>	Silently ignores the error that would occur if the window were closed while this loop is still scheduled.
173	<code>self.after(60, self._float_clouds)</code>	Reschedules this same method to run again in 60 milliseconds, producing continuous animation without blocking the rest of the UI.

5.5 Building the Storybook Panel (lines 178–280)

This is the largest method in the program: it lays out every interactive control inside the rounded card. It is presented here in logical groups rather than one continuous table, since most lines configure widget appearance rather than program logic.

5.5.1 Card container and title (lines 178–192)

Line	Code	Explanation
179–181	<code>self.panel_frame = tk.Frame(self, bg=SKY_TOP) / self.panel_frame.place(...)</code>	Creates an invisible frame centred in the window, sized to 88% of the window's width and 86% of its height, that will host the visible card.
183–186	<code>card = RoundedCard(...) / card.pack(...) / inner = card.inner</code>	Creates the rounded-rectangle card (see Section 5.7) inside <code>panel_frame</code> , then keeps a reference to its inner content frame, <code>inner</code> , where all real widgets are placed.
189–190	<code>tk.Label(inner, text="■ Ghibli Garden ■", ...)</code>	The large decorative title label at the top of the card.
191–192	<code>tk.Label(inner, text="a little spell for a strong password", ...)</code>	An italic subtitle/tagline directly beneath the title.

5.5.2 Password display ("scroll") (lines 194–207)

Line	Code	Explanation
195–198	<code>display_wrap = tk.Frame(inner, bg=PANEL_EDGE) / display_inner = tk.Frame(display_wrap, bg="#ffffdf5")</code>	Nests two frames — a thin tan-coloured outer frame and a slightly smaller cream inner frame — so a 2-pixel border appears around the password field, echoing an illustrated scroll or parchment.
200–204	<code>self.pw_entry = tk.Entry(..., textvariable=self.password_var, font=FONT_MONO, ..., state="readonly")</code>	The Entry widget that actually shows the generated password, centred, in bold monospace, and read-only so the user cannot accidentally edit the generated value.
206–207	<code>tk.Label(inner, textvariable=self.copied_var, ...)</code>	A small caption label bound to <code>copied_var</code> ; empty until the Copy button sets a confirmation message.

5.5.3 Strength meter widgets (lines 209–219)

Line	Code	Explanation
210–212	<code>self.strength_canvas = tk.Canvas(inner, height=14, ...) / .pack(...)</code>	A thin, full-width canvas that will later be filled with 1–4 coloured segments representing password strength.
213	<code>self._last_strength = (0, "#cccccc")</code>	Remembers the most recently computed strength level and colour, so the bar can be redrawn correctly if the canvas is resized before a new password is generated.
214–216	<code>self.strength_canvas.bind("<Configure>", lambda e: self._draw_strength_bar(*self._last_strength))</code>	Ensures the strength bar is redrawn at the correct width whenever the canvas' size becomes known or changes (see Section 5.6.3 for why this matters).
217–219	<code>self.strength_label = tk.Label(inner, text="", ...)</code>	A caption label that will show text such as "Totoro — big and dependable (~84 bits)".

5.5.4 Action buttons and divider (lines 221–230)

Line	Code	Explanation
222–223	<pre>btn_row = tk.Frame(inner, bg=PANEL) / btn_row.pack(...)</pre>	A horizontal row that holds the two action buttons side by side, centred beneath the strength meter.
224–225	<pre>LeafButton(btn_row, text="◆ Cast a New Spell", command=self.generate_password, ...)</pre>	Instantiates the custom LeafButton widget (Section 5.8) wired to call generate_password() when clicked.
226–227	<pre>LeafButton(btn_row, text="■ Copy", command=self.copy_password, ...)</pre>	A second LeafButton, in the rose colour scheme, wired to copy_password().
230	<pre>tk.Frame(inner, bg=PANEL_EDGE, height=1).pack(...)</pre>	A 1-pixel-tall frame used purely as a horizontal rule to visually separate the actions from the settings below.

5.5.5 Length slider (lines 232–247)

Line	Code	Explanation
233–236	<pre>len_row = tk.Frame(inner, bg=PANEL) / tk.Label(len_row, text="Length", ...)</pre>	A row containing the word “Length” on the left.
237–239	<pre>self.length_display = tk.Label(len_row, text=str(self.length_var.get()), ...) / .pack(side="right")</pre>	A second label, right-aligned in the same row, that echoes the currently selected numeric length.
241–246	<pre>self.length_scale = tk.Scale(inner, from_=6, to=48, orient="horizontal", variable=self.length_var, showvalue=False, ..., command=self._on_length_change)</pre>	The slider itself: it can select any integer length between 6 and 48, is bound to length_var (so moving it updates the variable automatically), hides Tkinter's own built-in numeric readout (showvalue=False, since the app draws its own), and calls _on_length_change on every move.
247	<pre>self.length_scale.pack(fill="x", padx=30, pady=(2, 14))</pre>	Places the slider beneath the label row, stretched to the panel's width.

5.5.6 Ingredient checkboxes (lines 249–270)

Line	Code	Explanation
250–251	<pre>tk.Label(inner, text="Ingredients for the spell", ...)</pre>	Section heading above the checkboxes.
253–254	<pre>opts = tk.Frame(inner, bg=PANEL) / opts.pack(...)</pre>	A frame that will arrange the checkboxes in a two-column grid.
256–261	<pre>self._make_check(opts, ..., self.upper_var, 0, 0) / ... five calls in total</pre>	Creates the five checkboxes — Uppercase, Lowercase, Numbers, Symbols (each in a 2-column grid), and Avoid look-alikes (spanning both columns on its own row) — each delegated to the _make_check helper (Section 5.5.7).
263–265	<pre>for var in (...): var.trace_add("write", lambda *_: self.generate_password())</pre>	Attaches a listener to all five option variables (four checkboxes plus the ambiguous-character toggle) so that ticking or unticking any of them immediately regenerates the password with the new settings, without needing a separate “apply” step.
267–270	<pre>tk.Label(inner, text="Soot sprites keep this password only on your screen – nothing is sent anywhere.", ...)</pre>	A reassuring privacy note at the bottom of the card, styled as small italic-feeling caption text.

5.5.7 Checkbox factory helper (lines 272–280)

Line	Code	Explanation
272	<code>def _make_check(self, parent, text, var, col, row, colspan=1):</code>	A small factory method that avoids repeating the same Checkbutton configuration five times.
273–278	<code>cb = tk.Checkbutton(parent, text=text, variable=var, font=("Georgia", 10), ..., onvalue=True, offvalue=False, ...)</code>	Creates one checkbox styled to match the storybook theme (cream check colour, no visible border) bound to the given BooleanVar.
279	<code>cb.grid(row=row, column=col, sticky="w", padx=6, pady=4, colspan=colspan)</code>	Places the checkbox into the requested grid cell, left-aligned, with a small margin, optionally spanning multiple columns.
280	<code>parent.grid_columnconfigure(col, weight=1)</code>	Allows the given column to expand, so the two-column layout shares space evenly as the window is resized.

5.5.8 Length-change callback (lines 282–284)

Line	Code	Explanation
282	<code>def _on_length_change(self, _val=None):</code>	Called automatically by the Scale widget every time the user drags it; the value itself is ignored (<code>_val</code>) because it can be read from <code>length_var</code> instead.
283	<code>self.length_display.config(text=str(self.length_var.get()))</code>	Updates the numeric readout label next to the word “Length” to match the slider’s new position.
284	<code>self.generate_password()</code>	Immediately regenerates the password at the new length, so the displayed password always matches the current slider position.

5.6 Password Generation and Strength Logic (lines 289–377)

5.6.1 `generate_password()` (lines 289–335)

This is the security-critical heart of the program. It is examined here in full, essentially line by line.

Line	Code	Explanation
289–29 2	<pre>def generate_password(self): / length = self.length_var.get() / pools = [] / required = []</pre>	Reads the requested length from the slider's variable; pools will collect the character sets the user selected, and required will collect one guaranteed character from each selected set.
294–29 7	<pre>upper = string.ascii_uppercase / lower = string.ascii_lowercase / digits = string.digits / symbols = "!@#\$%^&*()-_+[]{};:,<.>?/"</pre>	Defines the four possible character classes: 26 uppercase letters, 26 lowercase letters, 10 digits, and a hand-picked set of common symbol characters.
299–30 3	<pre>if self.avoid_ambiguous_var.get(): / ambiguous = set("lI100oB8S5Z2") / upper = "".join(...) / lower = "".join(...) / digits = "".join(...)</pre>	If the “avoid look-alikes” option is ticked, this filters out characters that are easily confused with one another when read or typed (e.g. lowercase l vs. uppercase I vs. digit 1; uppercase O vs. digit 0), producing cleaner versions of the upper/lower/digit strings.
305–30 7	<pre>if self.upper_var.get(): / pools.append(upper) / required.append(secrets.choice(upper))</pre>	If uppercase letters are enabled, the full uppercase set is added to pools (used for filling the remaining length), and one random uppercase letter is immediately reserved in required, guaranteeing at least one uppercase character appears.
308–31 6	<pre>(same pattern for lower, digits, symbols)</pre>	Repeats the identical enable-and-guarantee pattern for lowercase letters, digits, and symbols, based on each corresponding checkbox.
318–32 2	<pre>if not pools: / self.password_var.set("") / self.strength_label.config(text="Pick at least one ingredient ■") / self._draw_strength_bar(0) / return</pre>	Handles the edge case where the user has unticked every character-set checkbox: instead of crashing, it clears the password field, shows a friendly prompt, resets the strength bar to empty, and exits early.
324	<pre>all_chars = "".join(pools)</pre>	Concatenates every selected character class into one combined pool used to fill the rest of the password.
325–32 6	<pre>length = max(length, len(required)) / remaining = length - len(required)</pre>	Ensures the final password is never shorter than the number of guaranteed characters (which could happen if, e.g., the slider is set very low but all four sets are enabled), then computes how many additional random characters are still needed.
327	<pre>body = [secrets.choice(all_chars) for _ in range(remaining)]</pre>	Fills the rest of the password by drawing remaining characters, one at a time, uniformly at random from the combined pool, using the cryptographically secure <code>secrets.choice</code> .
328	<pre>password_chars = required + body</pre>	Combines the guaranteed characters with the randomly filled body into one list.
329–33 0	<pre>rnd = secrets.SystemRandom() / rnd.shuffle(password_chars)</pre>	Because the guaranteed characters were appended first, without shuffling they would always appear at the start of the password. <code>secrets.SystemRandom().shuffle</code> performs a cryptographically secure in-place shuffle so their position is unpredictable too.

Line	Code	Explanation
331	<code>password = "".join(password_chars)</code>	Joins the shuffled list of single characters back into the final password string.
333–335	<code>self.password_var.set(password) / self.copied_var.set("") / self._update_strength(password, pools)</code>	Publishes the new password to the display field, clears any previous “copied” confirmation message, and recalculates the strength meter for the new password.

5.6.2 `copy_password()` (lines 337–343)

Line	Code	Explanation
337–340	<code>def copy_password(self): / pw = self.password_var.get() / if not pw: / return</code>	Reads the currently displayed password and does nothing if it is empty (for example, if no character sets are selected).
341–342	<code>self.clipboard_clear() / self.clipboard_append(pw)</code>	Uses Tkinter's built-in clipboard access to clear whatever was previously copied and place the current password on the system clipboard.
343	<code>self.copied_var.set("✓ copied to clipboard – like catching a falling star")</code>	Updates the confirmation caption beneath the password field so the user gets immediate visual feedback that the copy succeeded.

5.6.3 `_update_strength()` — entropy estimate (lines 348–362)

Line	Code	Explanation
348–350	<code>def _update_strength(self, password, pools): / pool_size = len(set("".join(pools))) / entropy = len(password) * math.log2(pool_size) if pool_size > 1 else 0</code>	Computes the number of distinct characters available across all selected pools (<code>pool_size</code>), then estimates the password's entropy in bits using the standard formula $\text{length} \times \log_2(\text{alphabet size})$ — the theoretical number of guesses (as a power of two) an attacker would need in the worst case.
352–359	<code>if entropy < 40: level, color, label = 1, ... / elif entropy < 60: ... / elif entropy < 80: ... / else: ...</code>	Buckets the numeric entropy into four qualitative Ghibli-themed strength levels: Soot Sprite (under 40 bits, fragile), Catbus (40–60 bits, sturdy), Totoro (60–80 bits, dependable), and Spirit of the Forest (80+ bits, nearly unbreakable) — each with its own bar colour.
361	<code>self.strength_label.config(text=f"{label} (~{int(entropy)} bits)")</code>	Updates the caption to show both the friendly label and the underlying numeric bit estimate.
362	<code>self._draw_strength_bar(level, color)</code>	Redraws the segmented strength bar to visually reflect the computed level.

5.6.4 `_draw_strength_bar()` (lines 364–377)

Line	Code	Explanation
365	<code>self._last_strength = (level, color)</code>	Caches the level and colour just computed so the bar can be redrawn identically if the canvas is resized later (see the binding in Section 5.5.3).
366–367	<code>c = self.strength_canvas / c.delete("all")</code>	Clears any previously drawn segments before redrawing.
368–370	<code>w = c.winfo_width() / if w <= 1: w = 460</code>	Reads the canvas's current pixel width; if the widget has not yet been laid out by the window manager (width still at Tkinter's placeholder value of 1), a sensible default of 460 is used instead so the very first draw is not a sliver.

Line	Code	Explanation
371–373	<pre>segs = 4 / gap = 6 / seg_w = (w - gap * (segs - 1)) / segs</pre>	Defines four equal segments with a small 6-pixel gap between them, calculating each segment's width so the four segments plus three gaps exactly fill the available width.
374–377	<pre>for i in range(segs): / x0 = i * (seg_w + gap) / fill = color if i < level else "#e6dfc9" / c.create_rectangle(...)</pre>	Draws each of the four segments left to right: segments whose index is below the current strength level are filled with the strength colour, and the remaining segments are drawn in a muted beige to show them as “unfilled”.

5.7 Helper Class: RoundedCard (lines 380–409)

Tkinter has no built-in rounded-rectangle widget, so this small helper draws one using a canvas and a smoothed polygon, and hosts the panel's real content in an ordinary Frame placed on top of it.

Line	Code	Explanation
383–384	<pre>def __init__(self, parent, bg_color, edge_color, radius=24, **kwargs): / super().__init__(parent, bg=parent["bg"], **kwargs)</pre>	Initialises the outer Frame, matching its background to the parent so no seam is visible around the rounded shape.
385–386	<pre>self.canvas = tk.Canvas(self, ...) / self.canvas.pack(fill="both", expand=True)</pre>	Creates an internal canvas that fills the whole card area and will contain the rounded-rectangle drawing.
387–389	<pre>self.bg_color = ... / self.edge_color = ... / self.radius = radius</pre>	Stores the fill colour, border colour and corner radius for later use in <code>_redraw</code> .
390	<pre>self.inner = tk.Frame(self.canvas, bg=bg_color)</pre>	Creates the frame that will actually hold the title, password field, buttons, slider and checkboxes — i.e. everything built in <code>_build_panel</code> .
391	<pre>self.canvas.bind("<Configure>", self._redraw)</pre>	Redraws the rounded rectangle (and repositions the inner frame) whenever the card's size changes.
393–396	<pre>def _redraw(self, event): / c.delete("shape") / w, h,r = event.width, event.height, self.radius</pre>	Removes the previous rounded rectangle and reads the card's new width and height from the resize event.
397–399	<pre>c.create_round_rect = self._round_rect / c.create_round_rect(c, 3, 3, w - 3, h - 3, r, fill=..., outline=..., width=2, tags="shape")</pre>	Attaches the <code>_round_rect</code> helper to the canvas instance for convenience, then draws the rounded card shape with a 3-pixel margin so its border is not clipped at the widget's edge.
400	<pre>c.create_window(w / 2, h / 2, window=self.inner, width=w - 10, height=h - 10)</pre>	Embeds the ordinary inner content Frame as a window item centred on the canvas, sized slightly smaller than the card so its content sits comfortably inside the rounded border.
402–409	<pre>@staticmethod / def _round_rect(canvas, x1, y1, x2, y2, r, **kwargs): / points = [...] / return canvas.create_polygon(points, smooth=True, **kwargs)</pre>	Builds a rounded-rectangle by listing the eight key corner-approach points of the rectangle (approaching and leaving each of the four corners) and asking Tkinter to draw them as a single smoothed polygon; <code>smooth=True</code> lets Tkinter round off the corners into curves instead of sharp angles.

5.8 Helper Class: LeafButton (lines 412–443)

Both action buttons are drawn from scratch on a small Canvas rather than using the plain `tk.Button`, so their shape (a fully rounded pill) and colour transitions can be styled freely.

Line	Code	Explanation
415–418	<pre>def __init__(self, parent, text, command, base_color, dark_color, width=190, height=46, **kwargs): / super().__init__(parent, width=width, height=height, ...)</pre>	Creates the canvas at a fixed pixel size (default 190×46) with no border, ready to be drawn on.
419–423	<pre>self.command = ... / self.base_color = ... / self.dark_color = ... / self.text = ... / self.w, self.h = width, height</pre>	Stores all the values needed to redraw the button in either its resting or hover colour at any time.
424	<pre>self._draw(base_color)</pre>	Performs the initial paint of the button in its normal (resting) colour.

Line	Code	Explanation
425–427	<pre>self.bind("<Enter>", ...) / self.bind("<Leave>", ...) / self.bind("<Button-1>", ...)</pre>	Wires mouse events: entering the button repaints it in the darker hover colour, leaving it repaints it back to the base colour, and a left click triggers <code>_on_click</code> .
429–430	<pre>def _draw(self, color): / self.delete("all") / r = self.h / 2</pre>	Clears any previous drawing and computes a radius equal to half the button's height, so the rounded ends will be perfect semicircles.
432–435	<pre>self.create_oval(...) / self.create_oval(...) / self.create_rectangle(...)</pre>	Draws a left semicircle, a right semicircle, and a rectangle spanning the space between them, in the same colour — together forming one seamless, fully rounded pill shape (this is a common trick for drawing “stadium” shapes with basic canvas primitives).
436–437	<pre>self.create_text(self.w / 2, self.h / 2, text=self.text, font=FONT_LABEL_B, fill="#ffffdf5")</pre>	Draws the button's label, centred, in bold cream-coloured text on top of the coloured pill.
439–443	<pre>def _on_click(self): / self._draw("#4d7573") / self.after(120, lambda: self._draw(self.base_color)) / if self.command: self.command()</pre>	Gives a brief visual “press” effect by flashing a darker teal colour for 120 milliseconds before returning to the base colour, then calls whatever function was passed in as command (<code>generate_password</code> or <code>copy_password</code>).

5.9 Program Entry Point (lines 446–448)

Line	Code	Explanation
446	<pre>if __name__ == "__main__":</pre>	Standard Python guard ensuring the following lines only run when the file is executed directly (not when imported as a module).
447	<pre>app = GhibliPasswordGenerator()</pre>	Instantiates the main application window, which runs its full constructor (Section 5.3) and builds the entire interface.
448	<pre>app.mainloop()</pre>	Starts Tkinter's event loop, which keeps the window open and responsive to clicks, key presses and resize events until the user closes it.

6. Feature Summary

- Cryptographically secure password generation via Python's secrets module, including a secure shuffle so guaranteed characters are not predictably placed.
- Adjustable password length from 6 to 48 characters via a slider.
- Independent toggles for uppercase letters, lowercase letters, digits, and symbols, each guaranteed at least one occurrence when enabled.
- Optional filtering of visually ambiguous characters (l, I, 1, O, 0, etc.).
- Live, colour-coded, four-segment strength meter with an estimated entropy in bits.
- One-click copy to the system clipboard with on-screen confirmation.
- A fully hand-painted Studio-Ghibli-inspired scene — gradient sky, sun, rolling hills, a tree and shrub, and continuously drifting clouds — built entirely from canvas primitives with no external image assets.
- A reusable rounded-card container and a reusable pill-shaped button, demonstrating custom widget composition in Tkinter.

7. Testing

The application was tested on a headless Linux environment using an Xvfb virtual display. The window was launched, allowed to render fully, and screenshotted to visually verify the sky gradient, hills, storybook card, and control layout. Simulated mouse clicks (via `xdotool`) were then used to exercise the “Cast a New Spell” and “Copy” buttons, confirming that a new password is generated, the strength meter updates correctly, and the clipboard confirmation message appears as expected. No runtime exceptions were observed during these tests.



Figure 1: Application window after generating a password and pressing Copy, showing the strength meter at its highest (“Spirit of the Forest”) level and the clipboard confirmation caption.

8. Conclusion and Future Scope

Ghibli Garden demonstrates that a security-focused utility does not need a purely utilitarian interface: the entire visual theme is produced with basic Tkinter canvas drawing, while the password logic itself follows standard security practice by relying on the `secrets` module rather than the general-purpose `random` module. The project met all stated objectives: it produces strong, configurable, and clearly explained passwords inside a distinctive, hand-crafted interface.

Possible future enhancements include:

- A password history panel (session-only, never written to disk) styled as a scroll of past “spells.”
- An optional passphrase mode generating memorable word-based passwords.
- Sound effects or subtle particle animation on generation, for extra whimsy.
- Packaging the script as a standalone executable using PyInstaller for easier distribution.

References

- Python Software Foundation — `secrets` module documentation, [docs.python.org](https://docs.python.org/3/library/secrets.html).
- Python Software Foundation — `tkinter` module documentation, [docs.python.org](https://docs.python.org/3/library/tkinter.html).
- Python Software Foundation — `string` and `math` module documentation, [docs.python.org](https://docs.python.org/3/library/string.html).